

Simple Sorts

Lec 08b

Selection and Insertion sorts

DRAFT

Overview (recap)

Consider an array of integers, and the goal is to sort into non-decreasing order (largest on the right)

We will consider “simple sorts”:

1. Bubble sort
- 2. Selection sort**
- 3. Insertion sort**

Selection sort: Structure

- The same structure as bubble sort, except
- On the inner loop:
 - we do not do swaps immediately,
 - Instead, track “the largest element seen so far”
 - Only swap the selected element at the end, and only if needed

Pseudo-code

A sketch of the code structure is:

```
sort(int[] A)
    int n = A.size
    for (int k = n-1 ; k > 0 ; k-- ) // outer loop
        int max=0;
        for (int j = 1 ; j <= k ; j++ ) // inner loop
            if ( A[j] > A[max] ) max=j
        // after inner loop
        if ( max < k ) <<swap A[k] and A[max]>>
```

Example

Suppose $A = [9, 8, 7, 7']$

9	8	7	7'
---	---	---	----

- First pass: Scan the first 3 elements and select that 9 is the largest and needs to move to the end
- Hence, swap 9 with 7' to give

7'	8	7	9
----	---	---	---

- Second pass: scans [7' 8] and selects 8, then swap 8 with 7 to give

7'	7	8	9
----	---	---	---

- Third pass: selects 7' but it equals 7 so no swap done

7'	7	8	9
----	---	---	---

- Observe this means selection sort is not stable !
- We could try

Selection sort: Analysis

- As for bubble-sort count
 - Usage of the comparison operators
 - Swaps
- We still have to do a comparison for each element of each pass
- The difference is that on each pass do at most one swap

Pass 1: $n-1$ comparison ; up to 1 swaps

Pass 2: $n-2$ comparison ; up to 1 swaps

...

Pass $n-1$: 1 comparison ; up to 1 swaps

Selection sort: Complexity

Pass 1: $n-1$ comparison ; up to 1 swaps

Pass 2: $n-2$ comparison ; up to 1 swaps

...

Pass $n-1$: 1 comparison ; up to 1 swaps

Hence, we have

- $\Theta(n^2)$ comparisons
 - Note this is for both best and worst
 - Selection sort is not adaptive
- But only $O(n)$ swaps
 - This might be useful in cases when swaps are expensive

Can selection sort be made stable?

9	8	7	7'
---	---	---	----

Exercise (offline)

For

We consider

7'	8	7	9
----	---	---	---

- First pass: Scan the first 3 elements and select that 9 is the largest to the end

7'	7	8	9
----	---	---	---
- Hence, swap 9 with 7' to give

7'	7	8	9
----	---	---	---
- Second pass: scans [7' 8] and selects 8, then swap 8 with 7 to give
- Third pass: selects 7' but it equals 7 so no swap done

• Observe this means selection sort is not stable!

Selection sort on linked lists

9	8	7	7'
---	---	---	----

Exercise (offline)

Implement on a linked list

7'	8	7	9
----	---	---	---

Is it stable?

7'	7	8	9
----	---	---	---

For

7'	7	8	9
----	---	---	---

We coSuppose $A = [9, 8, 7, 7']$

- First pass: Scan the first 3 elements and select that 9 is the largest and needs to move to the end
- Hence, swap 9 with 7' to give

Summary of selection sort

- PRO: Simple and easy-to-implement
- CON: It is $\Theta(n^2)$
 - This is slow – even at $n=10^6$ we would have troubles
- PRO: It is naturally stable
 - No unnecessary re-ordering of elements
- CON: It is not adaptive
 - Unless do modifications, it does not naturally take advantage of the common case if the input already being “nearly sorted”
- PRO: it can also be easily implemented on linked lists (even singly-linked)
 - Exercise (offline): convince yourself of this

Insertion sort: Structure

- In bubble and selection sort, we build up an “already sorted subarray” at the end of the array
- Insertion sort takes the opposite approach and builds up an “already sorted subarray” at the start of the array
- It takes elements one at a time from the rest of the array and then inserts them at the right place into the “already sorted subarray” at the start of the array

Insertion sort: Example

Start with $A = [6, 4, 8, 5, 8']$ then the first entry '6' is an "already-sorted subarray"

6	4	8	5	8'
---	---	---	---	----

We take the next entry '4' and scan backwards doing swaps, until find the correct place for it

So, swap 4 and 6 to give

4	6	8	5	8'
---	---	---	---	----

Now have to insert 8 into [4,5]. No swaps needed, so get

4	6	8	5	8'
---	---	---	---	----

Insertion sort: Example

4	6	8	5	8'
---	---	---	---	----

Now have to insert 5 into [4,6,8].

We move backwards doing swaps with 5

Swap(8,5)

4	6	5	8	8'
---	---	---	---	----

Swap(6,5)

4	5	6	8	8'
---	---	---	---	----

Since $5 \geq 4$ we do not need to swap and can stop the backwards scan, giving

4	5	6	8	8'
---	---	---	---	----

Last pass has to insert 8' into [4,5,6,8] but needs no work, so get

4	5	6	8	8'
---	---	---	---	----

Insertion sort: Stability

We started with $A = [6, 4, 8, 5, 8']$
and finished with $A = [4, 5, 6, 8, 8']$

- Insertion sort is stable as never need to move past identical elements on the backwards scan

Insertion sort: Complexity

Worst case: the array starts in reverse order

Pass 1: 1 comparisons ; 1 swaps

Pass 2: 2 comparisons ; 2 swaps

...

Pass n-1: n-1 comparisons ; n-1 swaps

This is an arithmetic series again

Hence, in worst case, we have

- $\Theta(n^2)$ comparisons and swaps

Insertion sort: Best case?

Suppose that the input array is already sorted.

Every backwards scan will stop immediately, and so only needs 1 comparison and no swaps

Best case is $O(n)$

This is much better than the worst case, showing that insertion sort is adaptive

GOT HERE

Complexity? Best and worst cases

- Worst case : the array is reverse ordered
 - every potential swap is needed
 - Numbers of comparisons and swaps are both $\Theta(n^2)$
- Best case: the array is already ordered – no swaps are needed.
 - However, every comparison is still performed
 - Overall complexity is still $\Theta(n^2)$

Adaptive sorting

- In practice we are often sorting arrays that are already “almost sorted”
- If a sorting routine is much faster when the array is “almost sorted” then it is called “adaptive”
 - Because it adapts well to “good inputs”
- Bubble sort is not adaptive – it does not take much advantage of “almost sorted” inputs

Generalise to Simplify

Instead of always doing

```
sort( int[] A )
```

Consider sorting a sub-array defined by the first and last indices

```
sort( int[] A, int L, int R )
```

This seems to make things more complicated but paradoxically makes the implementations much clearer.

It is a standard technique we will use frequently

Summary of insertion sort

- PRO: Simple and easy-to-implement
- CON: It is $\Theta(n^2)$
 - This is slow – even at $n=10^6$ we would have troubles
- PRO: It is naturally stable
 - No unnecessary re-ordering of elements
- CON: It is not adaptive
 - Unless do modifications, it does not naturally take advantage of the common case if the input already being “nearly sorted”
- PRO: it can also be easily implemented on linked lists (even singly-linked)
 - Exercise (offline): convince yourself of this

Exercises (offline)

- Give pseudo-code for insertion sort
- Give recursive versions of selection and insertion sort
- Consider how to implement them on linked lists
- Can you make selection sort become stable

Minimum Expectations

- Know the simple sorting algorithm and how they work on examples
- Be able to justify/prove the big-Oh family of their best/worst case behaviours
- Understand the notion of it being a “stable” sort, and which algorithms are stable or not
- Understand the notion of “adaptive” and which algorithms are adaptive or not
- Be able to write them in a recursive form